LICHEN Project

Analysis Report

1. Executive Summary

LICHEN is a Python-based tool that automatically generates antibody light-chain sequences from a given heavy-chain sequence. It uses deep learning to help researchers in antibody engineering, reducing manual effort in sequence design. This report covers the architecture, dependencies, challenges, limitations, and an improvement that was made.

2. Project Overview

Antibody research requires matching a heavy-chain sequence with a compatible light-chain. LICHEN automates this step so researchers can generate and evaluate multiple candidates quickly.

Main Goals

- Generate compatible light-chain sequences from a heavy-chain input
- Support antibody research and drug development
- Reduce manual effort in sequence design

3. How It Works

LICHEN follows a simple step-by-step pipeline:

Step 1 – Input: The user provides a heavy-chain sequence.

Step 2 – Validation: The input is checked for errors and formatted correctly.

Step 3 – Tokenization: The sequence is converted into a format the model can read.

Step 4 – Generation: The model produces candidate light-chain sequences.

Step 5 – Filtering: Optional tools score and rank the generated sequences.

Step 6 – Output: Results are returned as a table or CSV file.

4. Key Files

pretrained.py – Main entry point — where users interact with the tool.

tokenizer.py – Converts sequences into tokens the model can understand.

model.py / inference.py – Generate the actual light-chain sequences.

utils.py – Helper functions for filtering, ranking, and saving results.

cli.py – Allows the tool to be used from the command line.

5. Dependencies

- **PyTorch** – Runs the deep learning model.
- **Biopython** – Handles biological sequence data.
- **Pandas** – Manages and exports tabular results.
- **ANARCII** – Validates antibody sequence numbering.
- **Humatch** – Scores how human-like a sequence is.
- **AbLang2** – Ranks and evaluates sequences.
- **psutil** – Detects system resources for optimal performance.

6. Challenges Encountered

- Uses pyproject.toml instead of requirements.txt, which is less familiar to beginners.
- Model weights must be downloaded separately — not obvious from the setup guide.
- Virtual environment issues came up during initial setup.
- Optional dependencies (ANARCII, Humatch) caused confusion when missing.
- Missing dependencies caused runtime errors that were hard to trace.

7. Limitations

- Setup is complex due to multiple external dependencies.
- Error messages are not very helpful for new users.
- Minimal logging makes it hard to debug long-running jobs.
- No automated test suite is visible in the project.
- Dependency management could be more consistent.

Although the project is functional, these limitations may create challenges for new users, increase development effort, and reduce operational efficiency during large-scale usage. Addressing these areas would improve the overall user experience, maintainability, and reliability of the system.

8. Recommendations

- **Add structured logging** – Easier to monitor and debug during long runs.
- **Improve input validation** – Clearer error messages for incorrect sequences.
- **Add automated tests** – Ensures correctness and prevents future bugs.
- **Use type hints** – Makes the code easier to read and understand.
- **Support config files** – Makes experiments easier to repeat and share.
- **Optimise batch processing** – Improves speed for large-scale sequence generation.

9. Improvement Made

The project uses the psutil library but did not list it in pyproject.toml. This caused runtime errors after what looked like a successful installation.

Fix applied: psutil was added to pyproject.toml so it installs automatically with the project.

10. Setup Instructions

1. Clone the repository.
2. Run: pip install .
3. Download the model weights from the provided link.
4. Run the example workflow to confirm everything works.

11. Conclusion

LICHEN is a well-structured tool that solves a real problem in antibody research. Its clear pipeline design makes it easy to understand and extend. Key improvement areas are usability, testing, and dependency management. A packaging bug with psutil was identified and fixed. With a few targeted improvements, LICHEN can serve a wider research audience.